

Colang, NeMo Guardrails, and Pipecat

How each one delivers control to an LLM, compared with a Hermes agent skill · Prepared 2026-10-07

1. What Colang is

Colang is the domain-specific language used by NVIDIA NeMo Guardrails to define how an LLM-based conversational app is allowed to behave. You write rules in Colang, and the Guardrails runtime enforces them around the model at runtime. The core ideas:

- **User and bot message definitions.** You give canonical names to kinds of utterances, with example phrasings. The runtime matches an incoming user message to the nearest canonical form by embedding similarity, so you are writing intents, not regexes.
- **Flows.** A flow is a sequence of what happens when a canonical user message arrives: which bot message to reply with, which action or tool to call, which check to run. This is where the guardrail logic lives.
- **Actions.** Python functions the flow can invoke, such as a fact-check call, a retrieval step, or a sensitivity filter.

A minimal Colang 1.0 example that blocks a topic:

```
define user ask about politics
    "What do you think of the election?"
    "Which party should I vote for?"

define bot refuse politics
    "I can't discuss political topics."

define flow
    user ask about politics
    bot refuse politics
```

Anything matched to "ask about politics" gets the refusal instead of reaching the LLM. Flows can also run before the model sees the input (input rails), after it responds (output rails), or around retrieval and tool calls. Colang 2.0 is a larger redesign with an event-driven, Python-like syntax, parallel flows, and support for multimodal interactions, but the same concept: a scripting layer that sits between the user and the model and dictates which paths the conversation is permitted to take.

Relative to a Hermes agent skill, it is the opposite philosophy. A skill is prose instructions the model is trusted to follow. Colang is an external controller that intercepts messages and enforces the rules whether or not the model would have complied.

2. How enforcement is guaranteed: Hermes skill versus Colang

Short answer: a Hermes skill guarantees nothing. Colang guarantees the parts it intercepts, and nothing else. They sit at different layers.

Hermes skill: advisory text in the context window

Checked against the hermes-agent loader. A skill is a Markdown file. The system prompt carries an index of skill names and one-line descriptions. When the agent decides a skill is relevant, it calls `skill_view(name)`, which reads the file and returns the text as a tool result. From that point the instructions

are just more tokens in the prompt. Nothing checks afterward whether the model did what the skill said. If the mathcode skill says "never use background=true," the model can still pass background=true, and the runtime will execute it.

Compliance rests entirely on the model's instruction-following. That is why the skill is written with capitals, "CRITICAL," and "MUST": the author is trying to raise the probability of compliance, because there is no mechanism to force it. It is also why a skill can describe a check but cannot make one happen.

Hermes does have hard enforcement, but at the tool layer, not the skill layer. The terminal tool runs approval guards on dangerous commands before executing them, and the skill loader refuses skills disabled by the user or unsupported on the platform. Those are code paths the model cannot talk its way around. A skill cannot add new ones.

Colang: a controller outside the model

A Colang flow runs in the NeMo Guardrails runtime, which sits between the user and the LLM. When a user message arrives, the runtime classifies it against the canonical user messages, selects a matching flow, and executes that flow. If the flow says "bot refuse politics," the canned refusal is returned and the LLM is never called. If the flow says "run the fact-check action on the output," that Python function runs and can block the response. The model does not get a vote on any of that. For the paths a flow covers, enforcement is a guarantee in the ordinary software sense: the code runs regardless of the model.

Where Colang's guarantee ends

- The classification step is itself model-based. Canonical-form matching uses embeddings, and some rails ask an LLM to judge. A paraphrase the matcher misses falls through to the default path. The guarantee is "if matched, enforced," not "every bad input is caught."
- Flows only constrain what they mention. Anything with no flow goes to the LLM unguarded.
- Output rails that call an LLM as a judge inherit that judge's error rate.

Side by side

	Hermes skill	Colang flow
Where it lives	Inside the prompt	Outside the model, in the runtime
What enforces it	Model's willingness to comply	Runtime code executing the flow
Can the model bypass it	Yes, by ignoring it	Not for matched paths
Failure mode	Model deviates silently	Input not matched, falls through
Hard guarantees available	Only via separate tool-layer guards	Yes, for anything the flow intercepts
Expressiveness	Anything you can write in prose	Only what the flow language and actions support

To actually guarantee a review step in a pipeline such as the MathCode formalize-then-prove flow, a skill could not do it. You would need either a Colang-style flow that halts after formalization and waits for confirmation, or a tool-layer guard that refuses to launch the next command until a confirmation flag exists. Both are code, not instructions.

3. Colang dictates the flow around the model

Colang dictates the flow around the model, and decides when and with what the model gets called. The model is one step inside the flow, not the thing executing it. One user turn in NeMo Guardrails:

- 1 The user message arrives at the Guardrails runtime, not at the LLM.
- 2 The runtime matches it to a canonical user intent from your Colang definitions.
- 3 It looks up which flow continues from that intent and starts executing the flow's steps in order.
- 4 Each step is one of three things. A bot message, which the runtime emits directly. An action, which is a Python function the runtime calls. Or a generation step, where the runtime calls the LLM with a prompt it assembled.
- 5 The flow can inspect the LLM's output and route it: emit it, rewrite it, run an output rail on it, or discard it and send something else.
- 6 Only what the flow decides to emit reaches the user.

So the model is invoked when a flow step says to invoke it, with the context the flow passes in, and its answer is only final if the flow lets it be. The flow owns control; the model fills in content. Two consequences:

- When no flow matches the user's message, the default path is to call the LLM normally. "Dictates" is true for covered intents, and the uncovered ones behave like a plain chatbot.
- The flow controls the model's inputs and outputs, not its internals. Colang cannot stop the model from thinking a wrong thing, only from that thing reaching the user or triggering an action.

The contrast with the Hermes skill is the direction of control. In Hermes, the model reads the skill and decides what to call. In Guardrails, the flow decides what to call, and the model is one of the things it can call.

4. A Colang script for sentiment-tagged avatar responses (reconstruction)

This is a representative reconstruction, not the original script. The pattern comes from NVIDIA's avatar stack (ACE, Tokkio), where Guardrails sits in front of the LLM and the bot's reply carries a sentiment or emotion tag that the avatar renderer maps to a facial expression, gesture, or voice style. The script usually does three things: classify the user's mood, tag the reply, and route edge cases away from the LLM.

Colang 1.0 form

```
define user express frustration
  "This is not working"
  "I've tried three times already"
  "Why is this so hard"

define user express gratitude
  "Thanks, that helped"
  "Perfect, you're great"

define user ask general question
  "How do I reset my password"
  "What are your hours"

define bot respond empathetic
  "[sentiment: concerned] I'm sorry this has been frustrating. Let's sort it out together."

define bot respond warm
  "[sentiment: happy] Glad that helped!"

define flow handle frustration
  user express frustration
  bot respond empathetic

define flow handle gratitude
  user express gratitude
  bot respond warm

define flow tagged general answer
  user ask general question
  $answer = execute generate_answer(query=$user_message)
  $tag = execute classify_sentiment(text=$answer)
  bot say "[sentiment: {$tag}] {$answer}"
```

`generate_answer` and `classify_sentiment` are Python actions registered with the runtime. The first calls the LLM. The second runs a sentiment classifier, which can be a small model or a second LLM call. The bracketed prefix is the tag the avatar layer parses off before speaking the text.

Colang 2.0 form

The same idea in the syntax the newer ACE releases use:

```
flow user expressed frustration
  user said "This is not working"
  or user said "Why is this so hard"

flow bot respond empathetic
  bot gesture "concerned"
  bot say "I'm sorry this has been frustrating. Let's sort it out together."

flow main
  activate llm continuation
```

```
when user expressed frustration
  bot respond empathetic
```

In 2.0 the sentiment tag becomes an explicit bot_gesture or bot_posture event the avatar subscribes to, rather than a string prefix.

CV phrasing

Say what it controlled and what consumed the tag. For example: "Authored Colang flows for an NVIDIA ACE avatar that classified user intent, generated LLM responses behind input and output rails, and emitted per-response sentiment tags consumed by the animation layer for expression and voice selection." Then add the specifics only you know: how many intents, whether the classifier was a model or rules, and what the tags drove.

5. NeMo Guardrails: the runtime, and its two run modes

Guardrails is the runtime that sits in front of the LLM. The Colang script is the configuration that runtime reads. Two different things. Guardrails is a Python library and server that receives the user message, calls the LLM when needed, and returns the final reply. Colang scripts are files you write that tell Guardrails what to do at each point. On their own they execute nothing. Guardrails loads them at startup and interprets them. Analogy: Kubernetes is the running orchestrator and the manifest YAML is what tells it what to do. Guardrails is the orchestrator, Colang is the manifest.

The package is nemoguardrails, installed with `pip install nemoguardrails`. It can run two ways:

Library form, one process

You import it in your own Python code, point it at a config directory containing your Colang files and a `config.yml`, and call it directly. The runtime executes inside your process. If your app crashes, Guardrails goes with it. If Guardrails needs a dependency, your app has to install it.

```
from nemoguardrails import LLMRails, RailsConfig

config = RailsConfig.from_path("./config")
rails = LLMRails(config)
reply = rails.generate(messages=[{"role": "user", "content": "Hi"}])
```

Server form, two processes

You start Guardrails on its own with `nemoguardrails server`. It becomes a standalone program listening on a port. Your app is a different program, possibly in a different language, possibly on a different machine. To get a guarded reply, your app sends an HTTP request:

```
POST http://guardrails-host:8000/v1/chat/completions
{"config_id": "avatar", "messages": [{"role": "user", "content": "Hi"}]}
```

Guardrails receives that, runs the flows, calls the LLM, and sends the reply back in the HTTP response. Your app never imports the library. "Behind an API" means the only way in is through those HTTP endpoints. Your app does not see the Colang files, the runtime state, or the LLM credentials. It sees a URL that accepts messages and returns replies. Why you would pick the server form:

- The avatar front end can be written in anything, such as a C++ or Unreal client, and still use Guardrails.
- One Guardrails server can serve many clients and many named configs.
- You can update the Colang files and restart the server without touching the client app.

- It fits container deployments. In Kubernetes terms, Guardrails becomes its own pod with its own Service, and the avatar app's pod calls it by name.

The trade is latency and one more thing to deploy. A function call is microseconds. An HTTP round trip is milliseconds plus a network dependency. The server is just the library wrapped in a FastAPI app. Internally it does the same thing a direct library call does, so you can develop and test with the library form, then deploy the identical config as a server.

The `config.yml` next to the Colang files is where you name the LLM provider and model, enable built-in rails such as jailbreak detection or fact checking, and set the embedding model used for intent matching.

Naming

Everything above refers to NVIDIA NeMo Guardrails, the open-source toolkit at [NVIDIA/NeMo-Guardrails](#) on GitHub. NVIDIA uses "NeMo" for several things. NeMo is also the name of their larger framework for training and fine-tuning models, and there is a separate NeMo Guardrails microservice sold as part of NVIDIA NIM / AI Enterprise. The microservice is a packaged, supported deployment of the same open-source runtime in server form. On a CV, "NeMo Guardrails" reads as the open-source toolkit unless you specify the microservice.

6. Pipecat compared with Colang controllers

Different problem, not a competing answer to the same one. Pipecat (from Daily) is a framework for building real-time voice and multimodal agents. Its job is moving audio, video, and text through a pipeline fast enough to hold a live conversation. Guardrails and Colang are about policy: what the model is allowed to say and do.

What Pipecat does

You build a pipeline of processors that pass frames to each other. A typical voice agent:

```
transport in (WebRTC/WebSocket audio)
  -> voice activity detection
  -> speech to text
  -> LLM
  -> text to speech
  -> transport out (audio back to the user)
```

Each stage is a `FrameProcessor`. Frames are typed objects: audio chunks, transcription text, LLM tokens, TTS audio, control signals such as "user started speaking." Pipecat's value is in the plumbing: streaming every stage, interrupting TTS when the user talks over the bot, managing turn-taking, and connecting to dozens of STT, LLM, and TTS providers.

How concerns reach the LLM in each

- In Pipecat, the LLM is one processor in the chain. You shape its behavior with the system prompt, a context aggregator that builds the message history, and function-calling. Anything you want enforced you write as your own processor placed before or after the LLM stage, in Python, operating on frames.
- In Guardrails, the LLM is a step inside a Colang flow. Policy is declared in Colang, and the runtime decides whether and how the model is called.

So Pipecat gives you a place to put control logic, a processor slot, but no policy language. Guardrails gives you a policy language, but no media pipeline. Pipecat has no concept of intents, rails, or canonical forms. Guardrails has no concept of audio frames, interruption, or latency budgets.

They stack rather than compete

A common avatar or voice-agent architecture: Pipecat handles the transport, speech recognition, and synthesis, and the "LLM" stage in the Pipecat pipeline is actually a call to a Guardrails server, which in turn calls the real model. That is the server form from section 5, used for the reason given there: a non-Guardrails component reaching it over an API.

All three side by side

	Hermes skill	Colang / Guardrails	Pipecat
Concern	Task know-how	Conversation policy	Real-time media and turn-taking
Form	Markdown in the prompt	Declarative flow language plus runtime	Python processor pipeline
Enforcement	None, model's choice	Runtime executes flows	Whatever your processors code
LLM's role	The decision-maker	One step in a flow	One stage in a stream

If avatar work used both, the CV phrasing would be along the lines of: built the real-time voice pipeline in Pipecat, with NeMo Guardrails in the LLM stage enforcing Colang-defined response policy and sentiment tagging.